

Estructuras de Datos

Parte 1 — Complejidad algorítmica (Big O)

1.1 Clasifica la complejidad

1. Buscar una palabra en un diccionario físico, abriendo siempre por la mitad de las páginas que quedan.

R//O($\log n$)

Cada vez que buscamos descartamos masomenos la mitad de las páginas que quedan, por eso aunque el diccionario tenga muchas más páginas, la cantidad de pasos aumenta lentamente.

2. Revisar, uno por uno, cada carné en una caja de carnés de estudiantes hasta encontrar el que buscas.

R//O(n)

Porque toca ir revisando los carnés uno por uno hasta encontrar el que necesito, si hay más carnés me voy a demorar más revisándolos.

3. Sacar la primera carta de un mazo ya barajado.

R//O(1)

Solo necesitamos sacar la primera carta, no importa si el mazo tiene 10, 50 o 100 cartas, la acción que hacemos es la misma.

4. Comparar cada estudiante de un salón con cada uno de los demás estudiantes, para ver si algún par cumple años el mismo día.

R//O(n^2)

Cada estudiante se compara con muchos de los demás estudiantes, al aumentar la cantidad de estudiantes, las comparaciones aumentan aproximadamente como el cuadrado de la cantidad de estudiantes.

5. Consultar la hora en tu reloj.

R//O(1)

Mirar la hora toma prácticamente la misma cantidad de pasos sin importar la hora o la cantidad de datos que existan.

1.2 De la vida real al análisis

Ejemplo: Organizar una cantidad de cuadernos uno por uno en mi bolso, si tengo 10 cuadernos, reviso y guardo cada uno y si tengo 20 tengo que hacer el mismo proceso con cada cuaderno.

Justificación: Esta tarea es $O(n)$ porque el tiempo depende de la cantidad de cuadernos, si la cantidad de cuadernos se hace mayor, entonces tardo más en organizarlos.

1.3 ¿Cuál escalar mejor?

La Forma B: $O(n \log n)$.

Aunque para un conjunto pequeño la forma A pueda ser más rápida, cuando la cantidad de datos crece, $O(n \log n)$ aumenta mucho más lentamente que $O(n^2)$, por eso si no sabemos qué tan grande será n en el futuro, la forma B sería una opción más segura.

1.4 Verdadero o falso

a. Un algoritmo $O(1)$ siempre es más rápido en segundos reales que uno $O(n)$.

Falso, Big O no indica exactamente cuántos segundos tarda un algoritmo, indica cómo crece su tiempo cuando aumenta la cantidad de datos.

b. La notación Big O describe cómo crece el tiempo de ejecución a medida que crecen los datos, no el tiempo exacto en segundos.

Verdadero, Big O sirve para analizar cómo cambia el costo de un algoritmo cuando aumenta la cantidad de datos, no dice exactamente cuántos segundos tardará en ejecutarse.

c. Buscar por índice en un arreglo (`arreglo[5]`) es $O(n)$, porque hay que recorrer los primeros 5 elementos.

Falso, en un arreglo se puede acceder directamente a una posición usando su índice.

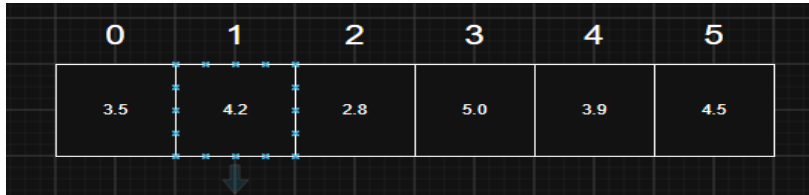
PARTE 2 — ARREGLOS 1D

2.1 Diseña el arreglo

Tenemos:

```
notas = [3.5, 4.2, 2.8, 5.0, 3.9, 4.5]
```

1. Dibujo del arreglo



2. Valor y índice tercera nota

La tercera nota es 2.8 y está en el índice 2.

3. Ultima nota

notas[5] porque hay 6 notas, pero como los índices empiezan desde 0, la última está en el índice 5.

2.2 Direcciones de memoria

1. Direcciones

$\text{dirección} = \text{base} + (\text{índice} \times \text{tamaño})$

notas[0]: Dirección: 0x2000

notas[3]: Dirección: 0x200C

notas[5]: Dirección: 0x2014

2. ¿Por qué notas[3] es igual de rápido que notas[0]?

Porque el computador puede ir directamente a la posición que le estoy pidiendo usando el índice, no tiene que pasar primero por las otras posiciones.

2.3 Búsqueda lineal vs. binaria

1. Búsqueda lineal

Se va revisando desde el primer elemento

15 no
18 no
20 no
23 no
27 no
31 sí

Hice 6 comparaciones, el 31 está en el índice 5.

2. Búsqueda binaria

Miro el elemento del medio (23) 15, 18, 20, 23, 27, 31, 35, 40 23 es menor que 31 entonces descarto la parte izquierda, queda: 27, 31, 35, 40 miro aproximadamente la mitad 27, 31, 35, 40 encontré el número.

Hice 2 comparaciones si cuento las comparaciones con los valores revisados: 23 y 31.

3. ¿Por qué la búsqueda binaria necesita que el arreglo esté ordenado?

Porque necesita saber qué parte puede descartar, por ejemplo, si veo que el número del medio es menor que el que estoy buscando puedo irme hacia la derecha porque sé que los números están ordenados, si estuvieran desordenados no sabría dónde buscar.

4. ¿Cuál usaría con 1 millón de elementos?

Usaría la búsqueda binaria, siempre y cuando el arreglo esté ordenado, la búsqueda lineal es $O(n)$ y puede tener que revisar muchísimos elementos, la binaria es $O(\log n)$ porque va descartando aproximadamente la mitad de los elementos en cada paso.

2.4 Insertar un elemento

1. Insertar al final

Si hay espacio disponible, sería fácil, solo pongo el nuevo elemento al final, pero si no hay espacio, tengo que crear un arreglo más grande y copiar los elementos.

2. Insertar 21 en la mitad

Para meter el 21, tengo que mover el 23 y el 27 una posición hacia la derecha para dejar espacio, por eso esta operación puede tomar más tiempo.

3. Comparación

Insertar al final con espacio: $O(1)$

Insertar al final sin espacio: $O(n)$

Insertar en la mitad: $O(n)$

PARTE 3 — ARREGLOS 2D

3.1 Diseña la matriz

Tenemos una matriz de **3 filas** $\times$ **4 columnas**.

1. Dibujo

	0	1	2	3
0	1	0	1	1
1	0	1	0	1
2	1	1	0	0

2. Acceder a fila 2 columna 3

`salon[2][3]`, el primer número indica la fila y el segundo indica la columna.

3. ¿Cuántas casillas tiene?

$3 \times 4 = 12$ Tiene 12 casillas

3.2 De 2D a memoria — Row-major

1. Dibujo para el cuaderno

1	0	1	1	0	1	0	1	1	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---

2. Dirección de [1][2]

Fórmula: dirección = base + ((fila × columnas + columna) × tamaño)

Reemplazamos:

$$= 0x1000 + ((1 \times 4 + 2) \times 4)$$

$$= 0x1000 + (6 \times 4)$$

$$= 0x1000 + 24$$

$$= 0x1018$$

Respuesta: 0x1018

3. Dirección de [0][0]

$$= 0x1000 + ((0 \times 4 + 0) \times 4)$$

$$= 0x1000$$

Respuesta: 0x1000

Dirección de [2][3]

$$= 0x1000 + ((2 \times 4 + 3) \times 4)$$

$$= 0x1000 + (11 \times 4)$$

$$= 0x1000 + 44$$

$$= 0x102C$$

Respuesta: 0x102C

3.3 Recorrido con ciclos anidados

1. Pseudocódigo

Ocupados $\leftarrow$ 0

PARA fila $\leftarrow$ 0 HASTA 2 HACER

PARA columna $\leftarrow$ 0 HASTA 3 HACER

```
SI salon[filas][columnas] = 1 ENTONCES
Ocupados ← Ocupados + 1
FIN SI
FIN PARA
FIN PARA
MOSTRAR ocupados
```

2. Complejidad

La complejidad es $O(f \times c)$, tengo que recorrer todas las filas y dentro de cada fila tengo que recorrer todas las columnas, por eso se multiplican f y c .

3. ¿Qué pasa si se duplican las filas y columnas?

Si se duplican las filas y las columnas, habría cuatro veces más casillas para revisar, por ejemplo si antes había $3 \times 4 = 12$ casillas, al duplicarlas habría $6 \times 8 = 48$ casillas.

3.4 Conectando todo

Big O sirve para saber cómo cambia el tiempo de un algoritmo cuando aumenta la cantidad de datos, los arreglos 1D sirven para guardar datos en una sola línea y los 2D permiten organizarlos en filas y columnas, depende del problema puedo usar uno u otro, también puedo escoger entre una búsqueda lineal o binaria dependiendo de cómo estén organizados los datos, por ejemplo la búsqueda binaria es más rápida cuando el arreglo está ordenado. Entender estos temas ayuda a elegir una forma más eficiente de trabajar con los datos.